

◆ Настройка окружения

Pinecone

Зарегистрируйтесь на pinecone.io – бесплатный тип (Starter) поддерживает один индекс и до 100 000 векторов, чего хватает для данной задачи.



После регистрации:

1. Создайте ключ API в разделе **API Keys**.
2. Сохраните его в файл `.env` в корне проекта (файл не комитировать - добавьте в `.gitignore`):

```
PINECONE_API_KEY = your-api-key-here
```

Python-зависимости

Создайте файл `requirements.txt`:

```
pinecone-client == 4.1 . 0
sentence-transformers == 3.0 . 1
rank-bm25 == 0.2 . 2
kaggle == 1.6 . 14
pandas == 2.2 . 2
numpy == 1.26 . 4
tqdm == 4.66 . 4
pyarrow == 23.0 . 1
python-dotenv == 1.0 . 1
```

Установка:

```
pip install -r requirements.txt
```

◆ Часть 1 — Подготовка данных и выбор инструментов

Цель: скачать датасет, выбрать и получить эмбединги.

1.1. Загрузка и подготовка датасета

Загрузите датасет через Kaggle (пригодится `~/.kaggle/kaggle.json`):

```
kaggle datasets download -d Cornell-University/arxiv
unzip arxiv.zip
# Результат: файл arxiv-metadata-oai-snapshot.json
# Формат: JSONL — каждая строка является самостоятельным JSON-объектом
```

Пример одной строки из файла (сокращенно):

```
{
  "id" : "0704.0001" ,
  "authors" : "C. Bal'azs, EL Berger, PM Nadolsky, C.-P. Yuan" ,
  "title" : "Сочетание множества дифференциальных производителей
кроссовок секций в "Тэватрон и " ,
  "
  LHC " "абстрактный" : "После дифференциальной четкости в твердом
количестве хромодинамических..." ,
  "versions" : [{ "version" : "v1" , "created" : "Mon, 2 Apr 2007
19:18:42 GMT" }],
  " 8-1 , 2-2
  "authors_parsed" : [[ "Baláz" , "C." , "" ], [ "Berger" , "EL" , ""
]]
}
```

Выполните скрипт.

```
# scripts/01_prepare_data.py
import json
import os
import pandas as pd
from tqdm import tqdm

INPUT_FILE = "archiv-metadata-oai-snapshot.json"
OUTPUT_FILE = "data/arxiv_subset.parquet"
MAX_RECORDS = 10_000

os.makedirs( "data" , exist_ok= True )

def extract_year ( paper: dict ) -> int :
    """
    Берем год из первой версии статьи – это дата публикации на arXiv.
    update_date – дата последнего обновления, она может быть на годы
    позже.
    Формат создан: "Mon, 2 Apr 2007 19:18:42 GMT"
    """
    try :
        versions = paper.get( "versions" , [])
        if versions:
            created = versions[ 0 ][ "created" ] # "Mon, 2 Apr 2007
19:18:42 GMT"
            # Год стоит на 4-й позиции после split по пробелу
            return int (created.split()[ 3 ])
    except:
```

```

except (IndexError, ValueError, Key
        формате "YYYY-MM-DD" return int (paper.get( "update_date" ,
"2000-01-01" )[: 4 ])

def format_authors ( paper: dict ) -> str :
    """
    authors_parsed – структурированный список [["Фамилия", "Инициалы",
"" ]].
    Собираем в читаемую строку "Фамилия И., Фамилия И."
    Если authors_parsed отсутствует – берем сырую строку authors.
    """
    parsed = paper.get( "authors_parsed" , [])
    if parsed:
        parts = []
        for entry in parsed[: 10 ]: # не более 10 авторов
            last = entry[ 0 ].strip() if len (entry) > 0 else ""
            first = entry[ 1 ].strip() if len (entry) > 1 else ""
            if last:
                parts.append( f" {last} {first} " .strip())
        return ", " .join(parts)
    # Запасной вариант: сырая строка авторов
    return paper.get( "authors" , "" ).replace( "\\n" , " " )

records = []
with open (INPUT_FILE, "r" , encoding= "utf-8" ) as f:
    для линии в tqdm(f, desc= "Читаем датасет" ):
        if len (records) >= MAX_RECORDS:
            break
        line = line.strip()
        if not line:
            continue
        paper = json.loads(line)

        abstract = paper.get( "abstract" , "" ).strip()
        title = paper.get( "title" , "" ).strip()

        # Пропускаем записи без аннотации или
        заголовка if not abstract or not title :
            continue

        # categories может содержать несколько категорий через пробел:
        "cs.LG cs.AI"
        # Берем первую как основную

```

```

categories_raw = paper.get( "categories" , "unknown" )
primary_category = categories_raw.split()[ 0 ]

records.append({
    "id" : paper[ "id" ],
    "title" : title.replace( "\\n" , " " ).strip(),
    "abstract" : abstract.replace( "\\n" , " " ).strip(),
    "authors" : format_authors(paper
    ), " year " : extract_y
    primary_category,
})

df = pd.DataFrame(records)
print ( f"\\nЗагружены статьи: { len (df)} " )
print ( f"\\nРаспределение по категориям (топ-10):" )
print (df[ "category" ].value_counts().head( 10 ))
print
( f " \\ nРаспределение по ' ].value_counts().sort_index().tail( 10
))
print ( f"\\nПример записи:" )
print (df.iloc[ 0 ].to_dict())

df.to_parquet(OUTPUT_FILE, index= False )
print ( f"\\nСохранено в {OUTPUT_FILE} " )

```

1.2. Выбор инструментов

В этой задаче используется **Pinecone** как векторная база данных и **allenai/specter2_base** модель эмбедингов.

В README письменно ответьте на следующие вопросы (не менее абзаца на каждый):

1. Чем Pinecone отличается от Qdrant и Chroma по модели развертывания, лицензии и производительности? В каком сценарии вы бы выбрали каждый из них?
2. Почему для задачи поиска по научным текстам выбрана модель specter2_base , а не универсальная all-MiniLM-L6-v2 ? Найдите карту модели на HuggingFace и процитируйте, для каких задач она обучена.
3. Что написано в карточке модели о рекомендованной метрике сходства? Почему это важно при создании индекса?

1.3. Получение эмбедингов

Вам необходимо реализовать скрипт 02_embed.py для преобразования текстовых данных в векторные представления (эмбединги) с использованием предварительно обученной

модели из HuggingFace.

Скрипт должен выполнять следующие шаги:

1. Загрузить датасет из файла `data/arxiv_subset.parquet` с использованием библиотеки `pandas`.
2. Подготовить тексты для кодирования:
 - для каждой записи объединить поля `title` и `abstract` в одну строку в формате: `title + " [SEP] " + abstract`
 - важно: токен `[SEP]` обязателен, поскольку модель обучена работать именно с таким форматом входных данных.
3. Сгенерировать эмбединги текстов с помощью модели `allenai/specter2_base` из библиотеки `sentence-transformers`.
4. Закодировать все тексты в эмбединги с учетом следующих требований:
 - использовать батчевую обработку (например, `batch_size=64`);
 - включить отображение прогресса;
 - нормализовать эмбединги (`normalize_embeddings=True`).
5. Вывести в консоль:
 - общее количество обработанных текстов;
 - размерность эмбедингов (ожидается 768);
 - норму первого эмбединга (должна быть близка к 1.0).
6. Сохранить полученные эмбединги в файле `embeddings/embeddings.npy` в формате NumPy.
7. Перед сохранением убедиться, что директория `embeddings` существует; при необходимости создать ее.

```
# scripts/ 02 _embed.py
!!! МЕСТО ДЛЯ ВАШЕГО КОДА!!!
```

В файле README ответьте на следующий вопрос:

Объясните, почему при использовании нормализованных эмбедингов (единичной длины) косинусное сходство (`cosine similarity`) эквивалентно скалярному произведению (`dot product`)?

◆ Часть 2 — Загрузка данных и метаданные

Сначала создайте индекс в Pinecone, а затем напишите скрипт `03_load_to_pinecone.py`, загружающий подготовленные эмбединги научных статей в этот индекс.

Скрипт должен выполнять следующие шаги:

1. Создать индекс `arxiv-papers` в Pinecone, если он еще не существует и подключиться к нему.
2. Загрузить данные:
 - прочитать датасет из файла `data/arxiv_subset.parquet` ;
 - загрузить эмбединги из файла `embeddings/embeddings.npy` .
3. Подготовить данные для загрузки:
 - обрабатывать записи батчами (например, по 200 элементов);
 - для каждой записи сформировать объект из:
 - уникальным `id` видом `"paper_<номер>"` ;
 - эмбедингами;
 - метаданными: `arxiv_id` , `title` , `abstract` (до 500 символов), `authors` (до 200 символов), `year` , `category` .
4. Загрузить данные в Pinecone батчами и показывать прогресс.
5. По завершении загрузки вывести в консоль общее количество векторов в индексе.

```
# scripts/03_load_to_pinecone.py
import os
import numpy as np
import pandas as pd
from tqdm import tqdm
from dotenv import load_dotenv
from pinecone import Pinecone, ServerlessSpec

load_dotenv()

INPUT_PARQUET = " data/arxiv_subset.parquet
"
INPUT_EMBEDDINGS
= " embeddings / embeddings.npy
"

# Инициализация клиента
pc = Pinecone(api_key=os.environ[ "PINECONE_API_KEY" ])

# Создаем индекс (если не существует)
!!! МЕСТО ДЛЯ ВАШЕГО КОДА!!!
```

Почему abstract обрезается до 500 символов?

Pinecone ограничивает суммарный размер метаданных одного вектора до 40 KB. Полный текст аннотации следует хранить отдельно (в parquet-файле) и подтягивать по ID после поиска.

◆ Часть 3 — Поисковые запросы

Напишите скрипт `04_search.py`, который выполняет семантический поиск по индексу Pinecone и сравнивает результаты с локальными вычислениями.

Скрипт должен выполнять следующие шаги:

1. Подключиться к индексу `arxiv-papers` в Pinecone и загрузить модель `allenai/specter2_base`.
2. Реализовать функцию кодирования запроса в эмбединге.
3. Выполнить **чистый семантический поиск**:
 - задать запрос (например: `"teaching machines to recognize objects in pictures"`);
 - получить топ-5 наиболее релевантных статей;
 - вывести результаты с названием, категорией, годом и частью абстракта.
4. Выполнить **поиск с фильтрацией**:
 - пример A: статьи `reinforcement learning` за последние 5 лет и категория `cs.LG`;
 - пример B: более старые статьи (до 2015 г.), любая категория;
 - сравнить выдачу и объяснить отличия.
5. Сравнить разные метрики сходства на локальных эмбедингах:
 - скачать все эмбединги из `embeddings/embeddings.npy`;
 - для заданного запроса вычислить:
 - cosine similarity;
 - dot product;
 - L2-distance;
 - вывести топ-5 статей для каждой метрики и сравнить результаты.

В файле README ответьте на обязательные теоретические вопросы:

1. Совпадают ли топ-5 для cosine и dot product и почему?
2. Отличаются ли результаты для L2 и почему?
3. Что бы произошло, если бы эмбединги не были нормализованы?


```
# scripts/ 04 _search.py
import os
import numpy as np
import pandas as pd
from dotenv import load_dotenv
from pinecone import Pinecone
from sentence_transformers import SentenceTransformer

load_dotenv ()

INDEX_NAME = "archiv-papers"
MODEL_NAME = "allenai/specter2_base"
TOP_K = 5

pc = Pinecone (api_key=os.environ[ "PINECONE_API_KEY" ])
index = pc. Index ( INDEX_NAME )
model = SentenceTransformer ( MODEL_NAME )
df = pd.read_parquet( "data/arxiv_subset.parquet" ) # для получения
полного abstract

!!! МЕСТО ДЛЯ ВАШЕГО КОДА!!!
```

◆ Часть 4 - Chunking

Хотя аннотации статей относительно кратки, некоторые из них превышают 512 токенов – максимум большинства sentence-transformer моделей. Кроме того, chunking — базовый навык для работы с реальными документами (PDF, книги, отчеты), где тексты гораздо длиннее.

Напишите скрипт 05_chunking.py, который разбивает длинные аннотации статей на чанки, создает эмбединги для каждого чанка и загружает их в отдельные индексы Pinecone для поиска по частям текста.

Скрипт должен выполнять следующие шаги:

1. Выбрать 30 статей с самыми длинными инструкциями.
2. Разбить тексты на чанки двумя стратегиями:
 - **Fixed-size chunking**: фиксированное количество слов с небольшим перекрытием между чанками;
 - **Semantic chunking**: объединение предложений до достижения максимального количества слов, чтобы сохранить содержание.
3. Создать отдельные индексы в Pinecone для каждого типа чанков (arxiv-chunks-fixed и arxiv-chunks-semantic).

4. Для каждого чанка:

- создать эмбединг с помощью модели `allenai/specter2_base`;
- сформировать объект с уникальным `id`, эмбедингом и метаданными: `arxiv_id`, `title`, текст чанка, номер чанка, год, категория.

5. Загружать чанки в Pinecone батчами и отображать прогресс.

6. Реализовать функцию поиска по чанкам:

- выполнить поиск по нескольким тестовым запросам;
- вывести топ-5 результатов для каждого типа чанков под названием статьи и частью текста чанка.

В файле README ответьте на обязательные теоретические вопросы:

1. Какая стратегия дает более осмысленные чанки?
2. Есть ли случаи разрезанных предложений и как это влияет на эмбединги?
3. Как размер `overlap` влияет на количество чанков и покрытие текста?

```
# scripts/ 05 _chunking.py
import os
import re
import numpy as np
import pandas as pd
from tqdm import tqdm
from dotenv import load_dotenv
from pinecone import Pinecone, ServerlessSpec
from sentence_transformers import SentenceTrans

load_dotenv ()

MODEL_NAME = "allenai/specter2_base"
VECTOR_DIM = 768

pc = Pinecone (api_key=os.environ[ "PINECONE_API_KEY" ])
model = SentenceTransformer ( MODEL_NAME )
df = pd.read_parquet( "data/arxiv_subset.parquet" )

!!! МЕСТО ДЛЯ ВАШЕГО КОДА!!!
```

◆ Часть 5 - Гибридный поиск

BM25 хорошо работает на точных терминах, аббревиатурах и именах авторов. Векторный поиск хорошо работает с перефразированием и синонимами. Гибридный поиск берет

лучше всего обоих.

Напишите скрипт `06_hybrid_search.py`, выполняющий гибридный поиск по корпусу статей, объединяя BM25 и векторное ранжирование с помощью Reciprocal Rank Fusion (RRF).

Скрипт должен выполнять следующие шаги:

1. Построить локальный BM25 индекс по заголовкам и аннотациям всех статей.
2. Подключиться к Pinecone и использовать модель `allenai/specter2_base` для поиска вектора.
3. Реализовать **Reciprocal Rank Fusion (RRF)** для объединения ранжированных списков BM25 и векторного поиска:
 - объединять результаты двух способов;
 - формировать общий топ-K документов.
4. Реализовать функции поиска:
 - BM25;
 - векторный (Pinecone);
 - гибридный (BM25+ векторный через RRF).
5. Для демонстрации выполнить три запроса:
 - точный термин (`"BERT fine-tuning"`);
 - имя автора (`"Yann LeCun convolutional networks"`);
 - перефразирование без явных терминов (`"making computers understand human emotions from text"`).
6. Вывести результаты для каждого метода и сравнить:
 - топ-5 BM25;
 - топ-5 векторного поиска;
 - топ-5 гибридного поиска по RRF, включая RRF-скор.

В файле README ответьте на обязательные теоретические вопросы:

1. Какой способ дал наилучший итог и почему?
2. Есть ли документы в топ-5 гибридного поиска, которых нет у топ-5 отдельных методов и почему?
3. Как изменение параметра k в RRF влияет на выдачу (например, $k=60$ vs $k=1$)?

```
# scripts/ 06 _hybrid_search.py
import os
import math
import numpy as np
```

```
import pandas as pd
from dotenv import load_dotenv
from pinecone import Pinecone
from sentence_transformers import SentenceTransformer
from rank_bm25 import BM25

load_dotenv ()

INDEX_NAME = "arxiv-papers"
MODEL_NAME = "allenai/specter2_base"
TOP_K = 10      # берем шире, чтобы RRF мог переранжировать

pc = Pinecone (api_key=os.environ[ "PINECONE_API_KEY" ])
index = pc. Index ( INDEX_NAME )
model = SentenceTransformer ( MODEL_NAME )
df = pd.read_parquet( "data/arxiv_subset.parquet" ).reset_index(drop=
True )

!!! МЕСТО ДЛЯ ВАШЕГО КОДА!!!
```

◆ Часть 6 - Анализ и выводы

Напишите в README развернутые ответы (не менее абзаца на каждый пункт):

1. **Семантический поиск vs BM25.** Приведите конкретные примеры запросов по вашей работе, где каждый метод выиграл. Сформулируйте общее правило: для каких типов запросов следует предпочесть каждому из них?
2. **Воздействие размера чанка.** Что происходит с качеством поиска, если чанк слишком маленький (10–15 слов)? Если слишком велик (500+ слов)? Есть ли оптимальный размер или зависит от задачи?
3. **Неподходящая метрика.** Что бы произошло, если бы мы создали индекс Pinecone с метрикой `euclidean` (L2), но использовали модель, возвращающую нормализованные векторы? Обоснуйте ответ математически: выведите связь между L2 и cosine для единичных векторов.
4. **Ограничение Pinecone Starter.** С какими ограничениями бесплатного тира вы столкнулись (или могли бы столкнуться)? Как бы вы решили задачу, будь датасет не 10000, а 10 миллионов статей?

Требования к README

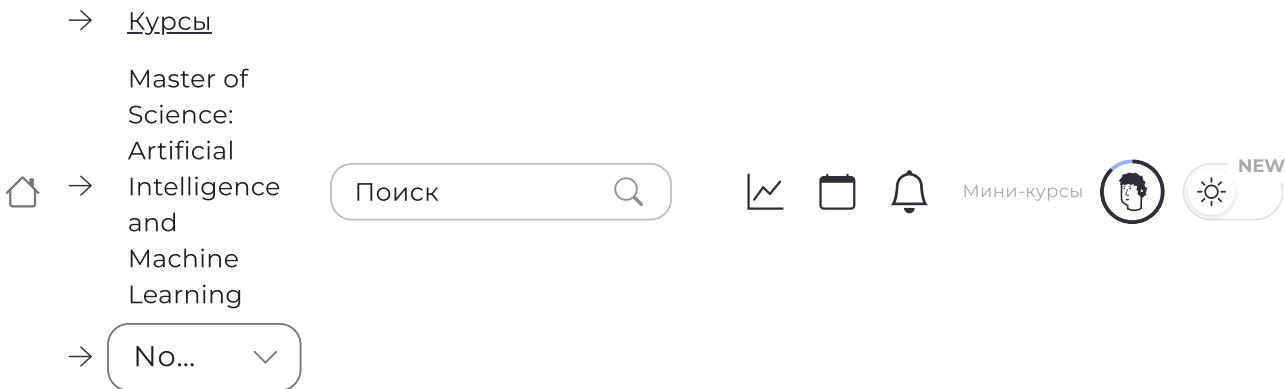
- письменные ответы на все вопросы из частей 1, 3, 4, 5, 6.
- вывод каждого скрипта (скриншот терминала или вставленный текст).
- сравнительная таблица методов поиска из части 5: три запроса × три метода.

Подготовка и загрузка задания

1. После выполнения всех этапов, скопируйте ссылку на ваш Git-репозиторий [Ваше
прізвище]_nosql_2 и прикрепите его в LMS.
2. Сохраните архив со всеми элементами работы и ответами на вопросы в файле
README на свой компьютер. Сохраненный архив должен также называться [Ваше
прізвище]_nosql_2 .
3. Прикрепите его в LMS в формате zip.
4. Отправьте задание на проверку.

Формат сдачи

- Прикрепленный архив с названием [Ваше прізвище]_nosql_2 .
- ссылки на Git-репозиторий с кодом, всеми элементами вашей работы и ответами на
вопросы (должны быть добавлены в README.md).



отправляет задания на доработку без оценки. Если вам «только уточнить» или вы застопорились на каком-то из этапов выполнения — обращайтесь к ментору в Slack.

Часть	Что проверяется	Вес
		Обзор ди
1	Данные загружены, обоснование выбора модели и БД аргументировано	10% Тема 1.
2	Индекс создан корректно, метаданы полные, батчевая загрузка работает	15% Научная
3	Все три типа поиска реализованы, результаты прокомментированы	25% Тема 2. Тема 3.
4	Обе стратегии chunking реализованы, сравнение осмыслено	20% Тема 4.
		Тема 5.

Часть	Что проверяется	✓ Вес
5	Гибридный поиск работает, RRF реализовано корректно, три примера показательны	✓ Научная 20% ✓ Тема 7.
6	Развернутый анализ компромиссов, математическое обоснование по метрикам	✓ Задание 10% ✓ Тема 8
		✓ Тема 9.
Часть	Что проверяется	✓ Задание Вес
1	Данные загружены, обоснование выбора модели и БД аргументировано	✓ Тема 10 10% Финальное
2	Индекс создан корректно, метаданы полные, батчевая загрузка работает	15%
3	Все три типа поиска реализованы, результаты прокомментированы	25%
4	Обе стратегии chunking реализованы, сравнение осмыслено	20%
5	Гибридный поиск работает, RRF реализовано корректно, три примера показательны	20%
6	Развернутый анализ компромиссов, математическое обоснование по метрикам	10%

Скрипт, падающий с ошибкой при запуске, не проверяется. Убедитесь, что все скрипты запускаются последовательно от 01_ в 06_ чистом окружении.

⚠ ВНИМАНИЕ!! У вас есть возможность выбрать подход к выполнению и возможной доработки задачи:

- удовлетвориться первой полученной оценкой (конечно же, если она выше проходного балла),
- пытаться получить высший балл путем возможной дальнейшей доработки работы в соответствии с фидбеком ментора.

Выбранный подход к выполнению ДЗ необходимо указать в поле для сдачи **к прикрепленному заданию. При отсутствии комментария ментор следует перво подходу и выставляет полученную оценку.**

Отправляйте задание на проверку, когда сделано все возможное, ведь количество попыток сдачи задания влияет на полученный балл! **За каждую последующую попытку после второй (то есть с третьей) максимально возможное количество баллов уменьшается на 5.** Таким образом: 1 попытка: максимальное количество баллов - 100, 2 попытка: максимальное количество баллов - 100, 3 попытка: максимальное количество баллов - 95, 4 попытка: максимальное количество баллов - и т. д. [Критерии оценки работ в магистратуре Golt Neoversity](#)

От работ ожидается освещение вашего собственного мнения, инсайтов, идей и выводов. Полностью сгенерированные тексты и другие работы с помощью ИИ без вашего личного вклада будут отправлены на доработку. Помните – ИИ разрешено использовать только как вспомогательный инструмент в процессе работы.



Задача ожидает выполнения



Дедлайн 23:45, 31 май.

Сдай домашнее задание!

Загрузи сюда ссылку на домашнее задание

B *I* U ~~S~~ {} ☰ ▼ ☷ ▼ 🔗 🔗



Загрузи файл до 150 Мб

Отправить домашнее задание